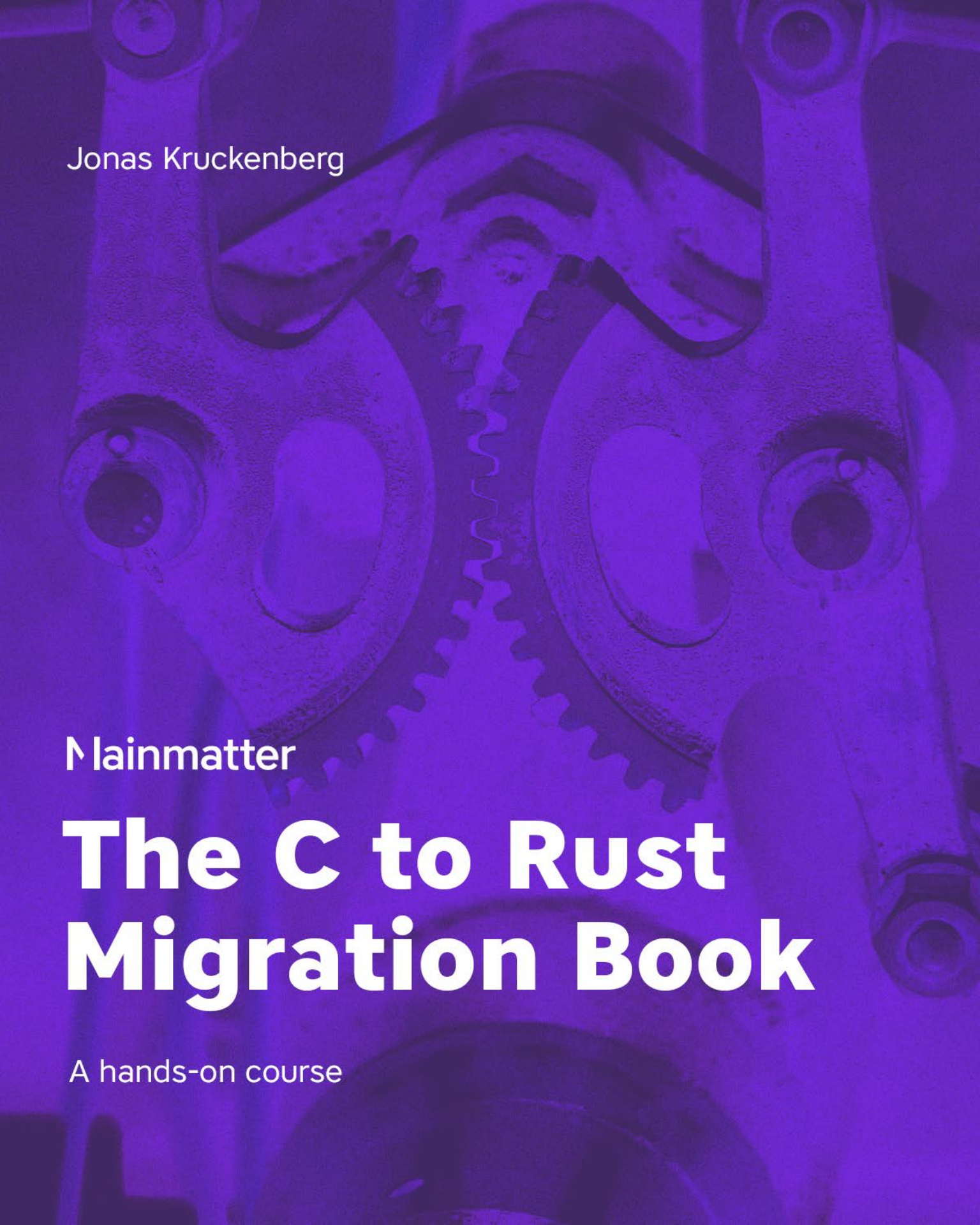




Jonas Kruckenberg

Mainmatter

The C to Rust Migration Book

A hands-on course

The C to Rust Migration Book

A hands-on course by Mainmatter

Jonas Kruckenberg

Contents

Welcome	1
1 Chapter 1: Basics	5
1.1 What Is FFI? How Do Languages Communicate?	5
1.2 When hand-written bindings drift	7
1.3 Generating Rust bindings with bindgen	9
1.4 Exposing Rust to C with cheadergen	10
1.5 extern "C" And FFI-Safe Types	12
1.6 The FFI boundary as a firewall: validate and narrow	15
1.7 High-quality FFI APIs	16
2 Chapter 2: Intermediate	19

Welcome

Welcome to the **“C to Rust Migration Book”**!

This course will teach you techniques for migration C codebases to Rust. You’ll learn how to maintain a safe mixed C-Rust codebase, incrementally migrate modules, translate common C idioms to Rust and learn the debugging tools to use when things inevitably break.

We assume prior knowledge of Rust and some C.

You’ll build up your knowledge in small, manageable steps. By the end of the course, you will have solved many exercises, and should be prepared to migrate even larger C codebases to Rust.

Methodology

This course is based on the “learn by doing” principle. It has been designed to be interactive and hands-on.

[Mainmatter](#) developed this course to be delivered in a classroom setting, over 4 days: each attendee advances through the lessons at their own pace, with an experienced instructor providing guidance, answering questions and diving deeper into the topics as needed.

If you’d like to organize a private session for your company, please [get in touch](#).

You can also take the course on your own, but we recommend you find a friend or a mentor to help you along the way should you get stuck.

Formats

You can go through the course material [in the browser]([\[TODO: insert URL\]](#)) or [download it as a PDF file]([\[TODO: insert URL\]](#)), for offline reading.

Structure

On the left side of the screen, you can see that the course is divided into sections. To verify your understanding, each section is paired with an exercise that you need to solve.

You can find the exercises in the [companion GitHub repository](#).

Before starting the course, make sure to clone the repository to your local machine:

```
git clone https://github.com/mainmatter/migrating-c-to-rust
```

We also recommend you work on a branch, so you can easily track your progress and pull in updates from the main repository, if needed:

```
cd c-to-rust-migration-book  
git checkout -b my-solutions
```

All exercises are located in the `exercises` folder. Each exercise is structured as a Rust package. The package contains the exercise itself, instructions on what to do (in `src/lib.rs`), and a test suite to automatically verify your solution.

Tools

To work through this course, you'll need:

[TODO: what's needed?]

- **Rust**. If `rustup` is already installed on your system, run `rustup update` (or another appropriate command depending on how you installed Rust on your system) to ensure you're running on the latest stable version.
- *(Optional but recommended)* An IDE with Rust autocompletion support. We recommend one of the following:
 - [RustRover](#);
 - [Visual Studio Code](#) with the [rust-analyzer](#) extension.
- A C compiler. The one provided by your Operating System will be good enough.

Workshop runner

To verify your solutions, we've also provided a tool to guide you through the course: the `wr` CLI, short for "workshop runner". Install `wr` by following the instructions on [its website](#).

Once you have `wr` installed, open a new terminal and navigate to the top-level folder of the repository. Run the `wr` command to start the course:

```
wr
```

`wr` will verify the solution to the current exercise.
Don't move on to the next section until you've solved the exercise for the current one.

We recommend committing your solutions to Git as you progress through the course, so you can easily track your progress and "restart" from a known point if needed.

Enjoy the course!

Author

This course was written by [Jonas Kruckenberg](#), Engineering Consultant at [Mainmatter](#).

Jonas Kruckenberg is a systems engineer and technologist focused on next-generation computing infrastructure. He is the lead author of k23, an experimental high-reliability operating system. As a TC39 Invited Expert, he helps shape the future of web by bringing non-browser perspectives to JavaScript language standardization.

Chapter 1

Chapter 1: Basics

In this chapter we'll learn the FFI foundations. We'll learn what FFI is, how we use it to interoperate between C and Rust, see common pitfalls and more!

Exercises

The exercises for this section are located in `exercises/01_intro`

Exercise

The exercise for this section is located in `01_intro/00_README` `\newpage{=latex}`

1.1 What Is FFI? How Do Languages Communicate?

Programming language design has taken many different paths over the years. We have C-like languages that are compiled all the way down to machine code, but we also have interpreted languages, lazy languages, languages that compile to a portable bytecode, garbage collected languages, and so much more.

As software engineers we do not want to duplicate work unnecessarily. So what happens when work exists in a language other than the one you're using?

When developing in - for example - Python, we don't want to port all the code we need to Python just to use it. Instead, we rely on infrastructure that lets us call functions and use types from other programming languages.

This infrastructure, which bridges different type systems (or lack of type systems), execution models, and code-organization concepts, is called a *Foreign Function Interface* (abbreviated *FFI* from here on out). Most modern languages have some way to interoperate with others through FFI. The exact syntax and options vary by language, but they almost always have one thing in common: they represent functions as if they were C functions.

This is called the “C ABI” (C Application Binary Interface) or “C Calling Convention” and as the name suggests is a convention on how to call functions (which CPU registers hold what arguments, which register(s) hold return values, when to spill to the stack, etc.)¹. Notice that this is just a convention that the industry agreed on over time. The C ABI is predictable, simple, and has been stable for decades and therefore became the de-facto standard for language interoperability.

Calling an FFI Function

Let’s say for example we have a Rust program that needs to call the `time` function from `libc` (a C static library)². We would use the following construct:

```
unsafe extern "C" {  
    fn time(time: *mut time_t) -> time_t  
}
```

Before we dissect the syntax though: you may have already noticed that nowhere in this snippet do we ask for `libc`. Why is that?

This is because object file formats (ELF on Linux, Mach-O on macOS, PE on Windows) are all quite old and therefore quite simplistic. They have *one global namespace* called a *Symbol Table* that all functions (and statics) share. So you cannot say “call `time` from `libc`”, you can only say “call a function named `time`”.

When a compiler builds a program, all function calls reference the function *by symbol* (“call function named `X`”). To make this actually executable, we need to replace every symbol reference with the actual address of the function. This happens *after* compilation during the *linking* step, where a separate program called the *Linker* will aggregate all object files that make up your final program, lay them out on disk and then resolve these symbol names.

To call `time` from `libc` we therefore need Rust to emit a reference to the `time` symbol and make sure the `libc` file is also passed to the linker.

This solves the problem of *what* to call, but we still need to figure out *how* to call that function: How many parameters does the function accept? What are the types of these parameters? How many return values does it return? Remember the symbol references above are plain string names. They do not carry any information about the function’s argument types or return types so Rustc has no way of figuring out the types itself.

This is why we need to tell it about `time`’s signature through a so-called “extern block” (or sometimes an “extern C block”) above. This block *declares* items that are not *defined* in the current crate. Each item we declare is a promise to the compiler: “this is the correct signature of this symbol, trust me”. We commonly refer to it as a **binding**.

Get the signature wrong and your Rust program will pass garbage to the FFI function without any way to check this at compile-time. The exact implementation won’t be known until link-time,

¹Technically, there is no “single” calling convention. Every architecture defines its own “C Calling Convention”. For example, the RISC-V C Calling Convention is defined [here](#) and lays out the sizes of C primitives and how arguments and return values are passed from and to functions. x86 architectures have *many* different calling conventions: the Microsoft x64 calling convention and the System V ABI are the most common, but many calling conventions exist to e.g. improve calling performance (`fastcall`, `regcall`, and more). When we say “the C calling convention” we usually mean “the C calling convention commonly used on this OS+architecture combination”.

²Yes, generally speaking `libc` is distributed not as a static but as a *dynamic* library which is a completely different way of linking and calling functions. We’ll cover this in a later chapter in detail.

much later than the compiler's type-checking pass. This is why bindings are marked unsafe: you as the programmer have to ensure signatures are correct.

This is the foundation of all Foreign Function Interfaces in Rust. In later exercises we will see how to make this much safer and more ergonomic.

Head to the exercise

Head to the exercise, where you'll write this block for a `bm_add` function implemented in C.

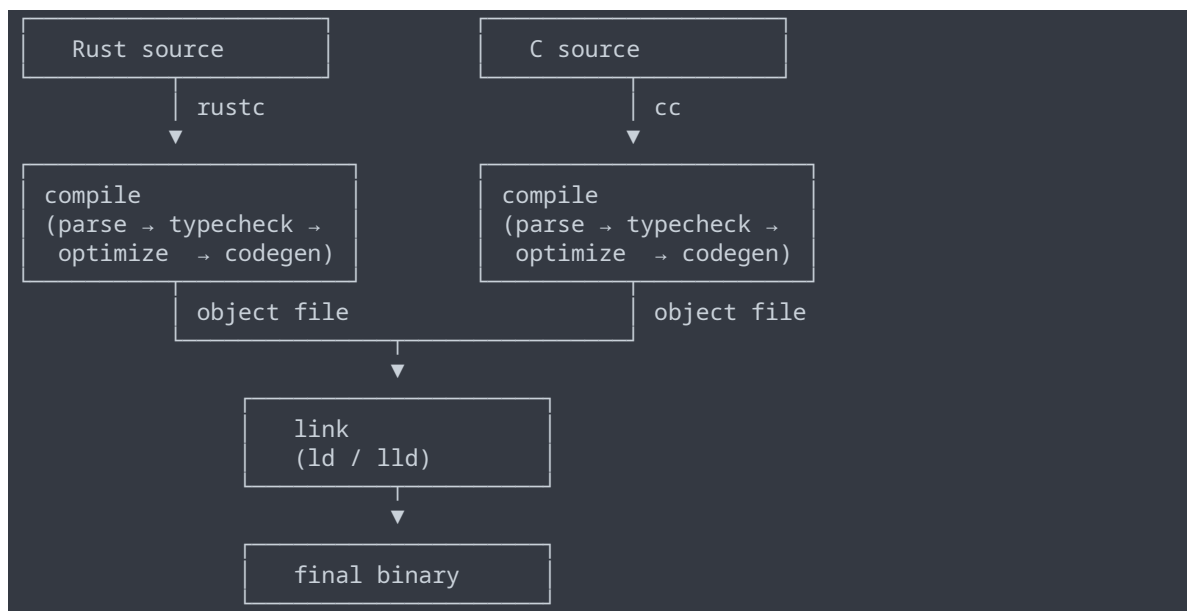
Exercise

The exercise for this section is located in `01_intro/01_what_is_ffi` \newpage{=latex}

1.2 When hand-written bindings drift

In the previous exercise you wrote an `extern "C"` block binding to C code. It worked great, but can you imagine yourself doing that for *hundreds*, maybe *thousands*, of functions and types? Of course not! There's a subtler problem, too. Hand-written bindings will *inevitably* drift out of sync with the C header, especially at scale.

Neither the C nor Rust compiler can detect this because each operate in their own small universe called a *compilation unit*. A compilation unit is the single chunk of work that flows through the various stages of the compiler. In C/C++ this would typically be a `.c / .cpp` file and in Rust that is typically a single crate. The compiler will parse, typecheck, optimize the compilation unit and produce an *intermediate* object file. Once all the compilation units that make up your project are built, the intermediate object files are gathered and passed to the *linker*, which produces the final executable or library.



This model is great for compilation performance because we can process many of these compilation units *in parallel*! There is a catch, though: compilation units *must not* share information since that would destroy our ability to process them in parallel! Each compilation *must* be its own self-contained universe.

And even if we had a mechanism to share information between compilation units, we would need to make that language agnostic so that a C compilation unit and a Rust compilation unit can interoperate. How would that even work with wildly different type systems? ³

So compilers have resorted to *manual escape hatches* like the `extern "C"` block you wrote. You as the programmer promise to the compiler that a function with given name and given signature will exist at link-time and the compiler takes your word for it.

Possible Consequences

The consequences of drift between the two versions can be dire: If C expects `int` but Rust expects `float` the integer bit patterns are being reinterpreted as floats. The result is almost always nonsensical.

It can be worse though: passing too many or too few arguments can result in either overwriting important information on the stack or reading garbage from the stack!

Here is an especially insidious case; the function as we have established is the following:

```
int bm_add(int a, int b);
```

but now in the Rust code, we expect it to return 64-bit integers instead of 32-bit integers:

```
unsafe extern "C" {
    fn bm_add(a: c_longlong, b: c_longlong) -> c_longlong;
}
```

How will this example fail? If you try this yourself you will notice: It doesn't! The reason is that modern CPUs pass function arguments in registers and these registers are always *word-sized*, meaning 64-bit on 64-bit machines. This means that internally even 32-bit ints are passed as 64-bit integers instead.

You can see this in Compiler Explorer here: [The correct version](#) passes `a` and `b` in the `a0` and `a1` registers (the `li a0, 1` and `li a1, 2` lines do the loading) and passes the return value in the `a0` register. Those registers are 64-bit registers though and so you can see the [incorrect c_longlong-expecting version just works](#) because the registers were 64-bit anyway.

Now, if your 64-bit numbers stay below the 32-bit max value, everything just happens to work. But it is very fragile. As you can see here, [when we compile to 32-bit target instead](#), everything breaks and we end up adding 0 to `a` instead.

I find this example particularly scary, because for 99% of inputs and deployment configurations this mistake will be virtually consequence-free. The addition will continue to work as expected.

³Yes the LLVM bitcode embedded by toolchains for LTO (Rust's `-Cltto=thin -Cembed-bitcode=yes` and Clang's `-flto=thin`) *does* carry the information to catch problems like this at link-time and it *is* cross-language, but linkers do not generally validate it (the `wasm-ld` linker does, but only for Wasm!). The reasons are manifold but most importantly because it would break existing compiler optimizations. You *could* write custom LLVM-bitcode parsing tooling to check this if you wanted.

But as soon as the input is unusual, the deployment target is different, *or you add code that will make the compiler change the generated code even a bit* this will be a bug that takes you weeks to troubleshoot in the worst case.

Head to the exercise

Head to the exercise and play with the different “drifted” C implementations of `bm_add`, see what happens on your machine. Feel free to also play around a bit with the Compiler Explorer playgrounds to see how different miscompilations manifest in the generated assembly.

Exercise

The exercise for this section is located in `01_intro/02_drift` \newpage{=latex}

1.3 Generating Rust bindings with bindgen

In the previous exercise you saw how easy it is for manual bindings to go out of sync, and the scary silent corruption that can cause. In this chapter we’ll introduce a tool to avoid all of this called `bindgen`. It’s a library you call in your Rust build script to automatically generate Rust bindings to C code based in C header files. It runs `libclang` on a C header and emits matching Rust declarations: `extern` blocks, `#[repr(C)]` structs, integer constants. The header remains the single source of truth, with the Rust bindings being generated on every build.

You call `bindgen` from your build script like so:

```
let bindings = bindgen::Builder::default()
    .header("c_src/bm_legacy.h")
    // tell Cargo to re-run when headers change
    .parse_callbacks(Box::new(bindgen::CargoCallbacks::new()))
    .generate()
    .expect("bindgen failed");

let out_path = PathBuf::from(env::var("OUT_DIR").unwrap());
bindings.write_to_file(out_path.join("bindings.rs")).unwrap();
```

This writes the generated file into the crate’s `OUT_DIR` (usually `target/<debug or release>/build/<crate name and hash>/out`). Inspecting the generated file, you will see something like this:

```
/* automatically generated by rust-bindgen 0.72.1 */

unsafe extern "C" {
    pub fn bm_add(a: ::std::os::raw::c_int, b: ::std::os::raw::c_int) -> ::std::os::raw::c_int;
}
```

You can then pull in this generated file into your crate by using the `include!` macro like so:

```
mod sys {
    include!(concat!(env!("OUT_DIR"), "/bindings.rs"));
}
```

This is great, we’ve gained end-to-end type checking. A change in the C header will not silently corrupt our Rust code. But remember these bindings are as unsafe as the C code itself. It’s your responsibility to use them correctly.

You would typically wrap them in a safe, idiomatic API. This is the common `-sys` crate pattern: a `<crate>-sys` crate holds the raw bindings, while the `<crate>` crate on top exposes the safe abstractions.

Head to the exercise

Replace the hand-written `extern` block from the previous chapter with `bindgen` in `build.rs`. Safe wrappers and tests stay identical.

Tip

`cargo expand -p bm_bindgen` shows what `bindgen` produced. Requires [cargo-expand](#).

Exercise

The exercise for this section is located in [01_intro/03_bindgen](#) \newpage{=latex}

1.4 Exposing Rust to C with `cheadergen`

So far, we’ve always called C from Rust. Now we flip. Remember that our goal is to *migrate* C code to Rust. We’ll eventually need C to call into our new Rust code!

To make a Rust function callable from C we’ll need to first annotate it correctly:

```
use std::ffi::c_char;

#[unsafe(no_mangle)]
pub extern "C" fn bm_strlower(s: *mut c_char) {
    // ...
}
```

Much like the FFI function declaration we saw in the first exercise, we instruct `rustc` to use a C-compatible calling convention through `extern "C" fn` (without it, Rust uses its own ABI that of course C knows nothing about).

`#[unsafe(no_mangle)]` keeps the symbol name verbatim in the compiled object. Otherwise Rust would emit a so-called *mangled* symbol name, a pseudo-random identifier that in regular Rust code prevents two equally named types or functions from causing “duplicate symbol” linking errors.

In this case we do want the symbol name to be the same plain identifier we gave it, so the C code linking to us can actually find the function.

The second piece is the header file that allows the C compiler to understand the API. You could write it by hand, but you'd be maintaining two sources of truth. **cheadergen** (the inverse of **bindgen**) generates C headers from Rust code.

cheadergen (created and maintained by fellow Mainmatter engineer Luca Palmieri) will read your Rust code and automatically generate a C header for the relevant FFI-visible symbols.

Install the CLI by following the instructions here <https://github.com/LukeMathWalker/cheadergen/releases>.

Then you can run the following command in your crate root (e.g. the current exercise crate):

```
cheadergen generate
```

and a header will appear:

```
/* Generated by cheadergen - do not edit */  
  
void bm_strlower(char *s);
```

cheadergen comes with two ways to configure its output. The first is a project-wide **cheadergen.toml** file. Here you can tweak the style of the generated headers, include custom preamble text, and more. See the full documentation [here](#).

The second configuration option is the `#[cheadergen::config(...)]` attribute. You can apply this attribute to any Rust type. It lets you configure per-type options such as renaming fields, skipping exporting, forcing exporting, and more. See the full documentation [here](#).

A note on **cbindgen**

If you've been researching Rust FFI tooling, you undoubtedly came across **cbindgen**, a *very similar* tool maintained by Mozilla, and you may ask yourself: "Why did you just introduce me to this custom tool instead?"

Well, we wouldn't have built a new tool if the existing one fit. **cbindgen** dates back to 2017, when the Rust ecosystem looked very different. To read your Rust and emit headers, it had to parse the source itself with a custom syn-based parser. At the time, that was the only way, since the compiler didn't expose enough information.

But the technique has real limits. The custom parser doesn't always agree with `rustc`, it often needs manual steering and fixups, and when it hits something it can't handle it tends to silently skip generating a header rather than tell you why. None of this is a knock on **cbindgen**: it's what the constraints of the era allowed. **cheadergen** exists because those constraints have since eased, letting us build something with better diagnostics and fewer surprises.

Head to the exercise

You'll port the `bm_strlower` function to Rust and call `cheadergen generate --lang c --output-dir c_test -p cheadergen` to generate the header automatically.

Exercise

The exercise for this section is located in `01_intro/04_cheadergen` \newpage{=latex}

1.5 extern "C" And FFI-Safe Types

We have passed data from C to Rust and from Rust to C in the previous exercises. In doing so we have mostly constrained ourselves to pointers or `c_`-prefixed primitives. At this point you may wonder though: “What if I want to pass a struct? or a String?”

Primitives

All primitives (i.e. `i8..i64`, `u8..u64`, `f32`, `f64`, `bool`, pointers) are always safe to share across an FFI interface since these correspond 1:1 with C types. `Option<NonNull<T>>` deserves special mention because it is *also* FFI safe and guaranteed to have the same representation as `*mut T`.

Note that C’s `int` type and Rust’s `i32`, for example, *almost always* mean the same thing. But because the C standard’s definition of these types is quite loose there exist architectures for which this is not true. This is rare enough that the Rust team decided Rust’s integer types are FFI-safe. If you want to be absolutely sure though, you can use the `std::ffi::c_*` types such as `c_int` or `c_longlong`.

repr(C)

Much like we need to use the C calling convention to make our Rust functions interoperable, we need to use the C struct layout to make our structs interoperable. This is because (just like functions) Rust reserves the right to change the struct layout at any time (it is “undefined”)⁴. In order to have a stable layout that other languages can understand though we need to mark our structs using the `repr(C)` attribute.

```
// we know the layout of this struct is always:
// - `a` - 8 bytes
// - `b` - 1 byte
// - 1 byte padding
// - `c` two bytes
#[repr(C)]
struct Foo {
    a: usize,
    b: u8,
    c: u16,
}

// we DO NOT know the layout of this struct!
struct Bar {
```

⁴The compiler does this to be smart and optimize things. For example, [here](#) is the code snippet you saw above in Compiler Explorer again. If you look at the rightmost “Compiler Output” pane you will see the actual layout of each struct. You can see that for `Bar` the compiler smartly reordered the fields to remove the interior padding byte!

```

    a: usize,
    b: u8,
    c: u16,
}

```

`repr(C)` also applies to *tuple structs* where the layout is exactly the same except the fields don't have names.

The `repr(..)` attribute can also be used on enums:

```

// This corresponds to named u8 constants, where A = 0, B = 1, C = 2
#[repr(u8)]
enum Foo {
    A,
    B,
    C,
}

// You can of course also assign explicit tags
#[repr(u8)]
enum Foo {
    A = 5,
    B = 2,
    C = 8,
}

// repr(C) also works and uses the "default enum size and sign for the target platform's C ABI"
#[repr(C)]
enum Foo {
    A,
    B,
    C,
}

```

You can use enums with fields even though they don't have an inherent C equivalent. Rust defines a stable mapping [here](#).

```

// A definition like this...
#[repr(u8)]
enum TwoCases {
    A(u8, u16),
    B(u16),
}

//...is in essence just syntax sugar for this:
union TwoCasesRepr {
    A: TwoCasesVariantA,
    B: TwoCasesVariantB,
}

#[repr(u8)]
enum TwoCasesTag {
    A,
}

```

```

    B,
}

#[repr(C)]
struct TwoCasesVariantA(TwoCasesTag, u8, u16);

#[repr(C)]
struct TwoCasesVariantB(TwoCasesTag, u16);

```

So essentially an enum with fields decomposes to a tag enum and a union of its fields.

repr(transparent)

`repr(transparent)` doesn't appear as often but warrants special mention. It is an attribute that can only be used on types with a single sized field. It guarantees that the layout of the outer type will be exactly the same as that of the inner type.

```

// Foo is guaranteed to have the same representation as `*const u8`!
#[repr(transparent)]
struct Foo(*const u8);

// because this is concerned with _sized_ fields (fields that have a size)
// fields that have no size such as PhantomData can still be used!
#[repr(transparent)]
struct Foo<T>(*const u8, PhantomData<T>);

```

`repr(transparent)` comes in handy if you need to cast pointers or transmute between types.

Types that cannot cross the FFI boundary

This list is long, but as a general rule of thumb: Any type with *generics* or more complex Rust structs that are not explicitly FFI-safe (such as `String`) cannot be shared.

Generics are not FFI-safe because the compiler will monomorphize a concrete version of the struct for each type passed into the generic. If we pass the type across the FFI boundary the C compiler (which does not know about monomorphization) would not know which version to pick! There exists no ABI that represents generics.

Head to the exercise

Head to the exercise. There you will find an FFI function that attempts to pass types that are not FFI-safe. Notice the compiler-generated warnings! It is your job to fix this by using FFI-safe types.

Exercise

The exercise for this section is located in [01_intro/05_ffi_safe_types](#) \newpage{=latex}

1.6 The FFI boundary as a firewall: validate and narrow

By now you’ve written `extern "C"` functions in both directions, seen `bindgen` generate Rust code and `cdeclgen` generate C headers.

On paper that is everything you need and it’s tempting to just go ahead and rewrite your project now. Take your C API and translate it one-for-one into Rust.

The problem is: often you can’t — and just as often, you shouldn’t. Real-world C is complicated, idiosyncratic, and (let’s admit it!) full of skeletons. You’re likely thinking about a Rust rewrite not just to address security problems or performance issues; rewriting in Rust is a chance to clean up your codebase: re-establish module boundaries, drop the legacy assumptions, write the module you wished you had.

At the same time big-bang rewrites (where you replace the entire codebase at once) famously never work. Which leaves a problem: you have a clean Rust codebase, a messy legacy codebase, and a need to keep both working together for *some time*⁵.

During this transitional period the FFI layer is where those worlds meet. It has to bridge *unsafe-everything-goes C* and the borrow checker — *and* bridge your new design and the old one.

One major misconception many people have is that FFI is about *type translation*. That is correct, of course, but it’s like saying programming is about typing words into a computer; it is missing the point. The *main* job of your FFI interface is **establishing confidence**. In a legacy codebase you rarely know with 100% certainty that reality matches your assumptions: some caller might actually pass a nullptr, the `*const c_char` that used to be UTF-8 might now hold binary data.

The FFI boundary is your chance to turn legacy uncertainty into known-good state before it crosses into your new system. Uncertainty leaks. Your FFI must be a vigilant firewall against it.

To this end we at Mainmatter have identified a number of rules that we nowadays live by.

Always validate your inputs

Validate every assumption at the FFI boundary. Null checks, length checks, UTF-8 checks. Panic or return an error if anything is even slightly off. In web servers the HTTP handler function is your primary interface to the “chaos world” of the internet, think of FFI functions the same way: they are your primary interface to the legacy-C “chaos world”.

You should also lean on Rust’s type system so “forgot to check” isn’t even an option. For example: at Mainmatter, we encourage contributors to use `Option<NonNull<T>>` instead of `*mut T` as much as possible, for the simple reason that a `*mut T` can be dereferenced directly (`*ptr`), which triggers UB if the pointer is null. You would have to add a manual `if ptr.is_null() {}` check before every pointer dereference. With `Option<NonNull<T>>` on the other hand, the type system *forces* you to handle the `None` case explicitly. Even the laziest `.unwrap()` will result in a loud panic instead of potentially silent UB.

⁵The exact amount of time of course varies with the scale of your codebase, but let me tell you from hard-won experience: measure this in years. C codebase migrations have a funny habit of taking much, much longer than you think. As a rule of thumb: take your worst case estimate, double that and add a year. Really.

```
// `*mut T` silently accepts null. You'll remember to check. Until you don't.
pub extern "C" fn bm_do_thing(input: *mut Thing) -> BmResult { /* ... */
}

// `Option<NonNull<T>>` encodes null as `None` — the compiler forces the branch.
pub extern "C" fn bm_do_thing(input: Option<NonNull<Thing>>) -> BmResult {
    let Some(input) = input else {
        return BmResult::ErrInvalid;
    };
    // `input: NonNull<Thing>`, statically non-null.
    /* ... */
}
```

Don't overload primitives

Legacy C code has a habit of packing several meanings into one return type. POSIX `read()` is a good example: `-1` means error (check `errno`), `0` means end-of-file, anything positive is the byte count. It is *very easy* to mess this up. Rust gives you better tools — `bool` for yes/no, an enum for branching outcomes, an out-parameter for the count. These types usually translate quite well into C headers too, so use them!

```
#[repr(C)]
pub enum BmReadStatus { Ok, Eof, IoError }

pub extern "C" fn bm_read(/* ... */, out_bytes: Option<NonNull<usize>>) -> BmReadStatus;
```

Head to the exercise

Head to the exercise. We'll continue with the porting work by taking a look at `bm_normalize_url`, found here `exercises/_bm/src/bookmark.c`. It will normalize a given URL by lowercasing it and writing the normalized string into the provided buffer. The exercise already contains our “*first draft*” Rust translation: a naïve transliteration from C to Rust that has a number of issues.

Let's apply what we learned above and fix the implementation!

Hint: check the exercises tests to check the expected behaviour of `bm_normalize_url`

Exercise

The exercise for this section is located in [01_intro/06_validate_and_narrow](#) \newpage{=latex}

1.7 High-quality FFI APIs

We previously introduced two “mechanical” rules: always validating your input and not overloading primitives. Now we will introduce two more rules that are a bit more conceptual but have proven themselves *very* useful.

But don't let that fool you, the following rules are just as important as the previous two, if not more so.

Documentation, documentation, documentation

Some invariants you can check at runtime. Many you can't — who owns this pointer, whether the string is copied or borrowed, whether free has run before, what each error variant means. Write them all down.⁶ Rust's # Safety convention (being just a comment) works with extern "C" functions too! `chadergen` will emit them as C doc comments in the header files:

```
/// Normalize a URL into the caller's buffer.
///
/// # Ownership
/// `url` and `out` are borrowed for the call; the caller continues to own both.
///
/// # Safety
/// 1. `url`, if non-null, must be a [valid] pointer to a NUL-terminated UTF-8 byte sequence.
/// 2. `out`, must be a [valid], non-null pointer to a writable buffer of at least `out_len` bytes.
///
/// # Errors
/// - `BmResult::ErrInvalidUrl` if `url` is null or not valid UTF-8.
/// - `BmResult::ErrBufferTooSmall` if the result wouldn't fit in `out_len` bytes.
///
/// [valid]:
#[no_mangle]
pub unsafe extern "C" fn bm_normalize_url(
    url: Option<NonNull<c_char>>,
    out: Option<NonNull<c_char>>,
    out_len: usize,
) -> BmResult {
    let Some(out) = out else {
        //...
    };

    // Safety: caller ensured `out` points to at least `out_len` bytes (2.)
    unsafe { slice::from_raw_parts_mut(out.as_ptr(), out_len) };
}
```

Note how we number safety invariants and force every inline safety comment to either:

- Delegate upholding its local invariant to the surrounding function's safety comment. In this case it MUST reference a numbered invariant
- OR it must exhaustively explain why the local invariant is upheld by the code itself.

This way we make sure that all invariants are either upheld by the function itself or correctly documented as a responsibility of the caller.

⁶You may (jadedly) say that no one ever reads comments and you may actually be correct. BUT, with the rise of LLMs *something* does actually "read" them. We've found that LLMs unsurprisingly struggle quite a bit with the nuanced, unspoken invariants of FFI code. Turning as many of these unspoken invariants into spoken ones helps you get better mileage out of these tools.

At the moment this is checked by discipline and PR reviews but in the future this may soon find its way into Rust/Clippy proper.⁷

Mind the FFI tax

Every exposed function is API surface you'll maintain forever, an unsafe contract to keep correct, and a per-call cost the compiler can't optimise away. Cross-language LTO can inline across, at the cost of a real setup burden. The cheapest FFI function is the one you didn't expose. Expose coarse operations — `bm_thing_update_with(...)` over one setter per field — and treat the boundary as a small set of verbs, not a mirror of your internal struct.

Head to the exercise

Head to the exercise where we will update our exercise 06 solution with the new rule(s) we have learned.

Exercise

The exercise for this section is located in [01_intro/07_high_quality_ffi_apis](#) \newpage{=latex}

⁷There are a couple of related proposals, all in the “pre-RFC” stage. The most interesting one being <https://github.com/safer-rust/safety-tags/blob/main/pre-RFC.md>

Chapter 2

Chapter 2: Intermediate

Note: the following chapters are **not yet publicly available!**

we're working hard on publishing the rest when we can. If you want early access, reach out!

From now on we will be porting a "real" C application: the `bm` bookmark manager CLI. We will learn how to approach real-world codebases, structure our approach, and debug our code when it breaks.

Exercises

The exercises for this section are located in `exercises/02_intermediate`

Exercise

The exercise for this section is located in `02_intermediate/00_README` \newpage{=latex}

Mainmatter

mainmatter.com/rust
info@mainmatter.com